



MATHEMATICAL MODELLING

EXTENSION OF INVESTIGATION OF GLIDER'S TRAJECTORY

ANIL KUTUWO
NEPAL

VU LE
VIET NAM

July 11, 2017

Summary

By extending the mathematical modelling cycle of a glider's trajectory, this work concentrated on two different ideas of mathematical model refinement. Firstly, a brief summary of previous work is presented to familiarise the readers of previous work conducted by the authors. The glider's mathematical model are provided with its system of four differential equations, as well as the numerical and analytical solutions are given to demonstrate the validity of the model. However, it is also apparent that based on the preliminary work, suggestions were made to promote different refinements of the provided model.

Chapter 3 introduces the relationship of two important values, which are coefficients of lift and drag. Aerodynamically speaking, these two variables are not constant, but depend on the value of angle of attack (glider's angle of orientation). Eventhough the information of this dependency can easily be obtained once the type of airfoil is known (NACA file), there is encountered difficulty in trying to replicate the relationship and still maintain a clear and conside relationship between the expressions of coefficients of lift and drag, and its equilibrium solutions. Therefore, despite being able to replicate better the actual glider's flight, it is observed that the equilibrium angle of orientation is much larger than the optimal's glider orientation, which should be closer to zero degree.

Subsequently, the effect of wind is studied by including a new term inside the differential equation of $\frac{dV}{dt}$. The term is used to describe the wind behaviour, and the sign is there to determined the direction of wind based on the glider's flight direction. Implementation of Euler's method by mean of Python program was applied to obtain solutions numerically. Using analytical method, the calculated solutions are verified and validated to be realistic and it was concluded that the wind should damp the oscillation behaviour of the glider's mathematical model.

Contents

Summary	3
List of variables	6
1 Introduction	7
2 Preliminary mathematical modelling of glider	8
2.1 System of governing equations of glider's mathematical model .	8
2.2 Euler's method and numerical simulation process	10
2.3 Phase plane and evaluation of mathematical model's result . .	13
2.4 Conclusion	14
3 Inclusion of dependency of lift and drag coefficients on angle of attack	15
3.1 Research on functions of lift and drag	15
3.2 Numerical simulation of glider's model	17
3.3 Interpretation of solution of mathematical model	20
3.4 Conclusion	20
4 Glider's behaviour during wind	21
4.1 Effect of wind during flight	21
4.2 Mathematical model considering wind effect	22

4.3	Simulation of glider system	22
4.4	Validation of mathematical model	24
4.5	Conclusion	25
5	Conclusions	26
	Bibliography	28
A	Preliminary simulation using Euler's method - Python programming code	29
B	Dependency of C_D and C_L on angle of attack simulation - Python programming code	34
C	Effect of wind on the flight of glider - Python programming code	39
D	Values of C_D and C_L against angle of attack of FX 66-17AII-182 airfoil	44

List of variables

θ	Angle of orientation of glider [rad]
θ_{eq}	Angle of orietation of glider at equilibrium point [rad]
m	Mass [kg]
t	Time [second]
V	Velocity of glider [m/s]
V_{eq}	Velocity of glider at equilibrium point [m/s]
g	Gravitational acceleration [m/s^2]
C_D	Drag coefficient [kg/m]
C_L	Lift coefficient [kg/m]

Chapter 1

Introduction

As an extension of the MathMod MOOC online course offered by TU Delft, the objective of this work is to further elaborate the basic governing system of differential equations to investigate a gliders flight trajectory. The work was conducted by a group of two students and two directions had been pursuit: the inclusion of lift and drag coefficients dependency on the gliders angle of attack; the effect of wind against the glider during its flight.

Preceding work had been done and evaluated to gain fundamental comprehension of the gliders mathematical model, as well as the meanings of all incorporated terms and their influences on the behavior of solutions of the system. Eulers method was implemented during the simulation process to numerically obtain solutions with sufficient precision. By applying different initial conditions and constants, analytical method was employed to effectively verify the validity of the solutions obtained from Eulers method, while visualization of solution was constructed by mean of a phase plane.

Following the suggestions proposed because of the previous work, this report will illustrate the results obtained from two separate refining ideas. First and foremost, Chapter 2 will briefly summarize the gliders mathematical model and what had been done during the previous project. Then, the inclusion of lift and drag coefficients dependency on angle of attack will be showed in Chapter 3 as well as the authors findings regarding the feasibility of this idea. Chapter 4, however, is responsible to demonstrate the effect of wind opposing the gliders motion during its flight. Eventually, a final chapter will conclude with the findings and discussions regarding each of the respective abovementioned ideas.

Chapter 2

Preliminary mathematical modelling of glider

The sole purpose of this chapter is to introduce the fundamental governing system of equations and the results obtained during the previous mathematical modelling cycle of a glider. Therefore, extending works in chapter 3 and 4 will be represented based on the observations made during the first work.

2.1 System of governing equations of glider's mathematical model



Figure 2.1: **Glider's trajectory problem reference framework.** (TU Delft, 2017.)

$$m \frac{dV}{dt} = -C_D V^2 - mg \sin(\theta) \quad (2.1)$$

$$mV \frac{d\theta}{dt} = -C_L V^2 - mg \cos(\theta) \quad (2.2)$$

$$\frac{dx}{dt} = V \cos(\theta) \quad (2.3)$$

$$\frac{dy}{dt} = V \sin(\theta) \quad (2.4)$$

Providing the basic idea to initiate the mathematical modelling process in the investigation of a gliders trajectory, a referencing framework is required for the formulation of necessary differential equations. The x and y axes represent horizontal and vertical position of the gliders center of mass.

Based on Newtons second law of motion, the equations 2.1 and 2.2 were derived to describe the two orthonormal force directions (along and normal to the direction of the gliders velocity). To calculate the position of the glider in space, the last two differential equations are formulated with respect to the velocity and angle of orientation of the glider.

With the given system of equations, mandatory information of a glider can be used to simulate the solutions using Eulers method in Python. Attached below are the specifications of a typical glider model, Cirrus 75.

Model	Cirrus 75
Number of seats	1
Airfoil, wing	FX 66-17 A II-182
Wing span	15 m
Wing area	10 sq m
Aspect ratio	22.5
Gross weight	390 kg
Maximum L/D	37
Typical speed	93 km/h

Table 2.1: **Cirrus 75 Glider specification.** (Standard Cirrus, n.d.)

Additionally, an essential assumption during this procedure was that coefficients of lift C_L and drag C_D were made to be constant. Though this simplification did not reflect the actual varying nature of these two non-dimensional values, it allows for rapid calculation of solutions both numerically and analytically which benefits the evaluation process.

2.2 Euler's method and numerical simulation process

The results of Eulers method are attached below for visualization purpose.

$$dV/dt = (-CdV^2 - mg\sin(\theta))/m, d\theta/dt = (CIV^2 - mg\cos(\theta))/(mV)$$

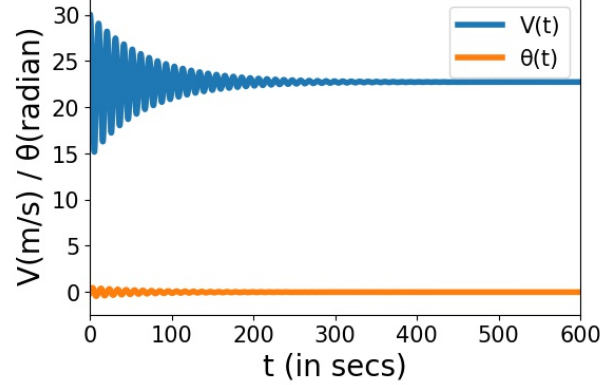


Figure 2.2: **Variations in velocity and angle with time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$

$$dV/dt = (-CdV^2 - mg\sin(\theta))/m, d\theta/dt = (CIV^2 - mg\cos(\theta))/(mV)$$

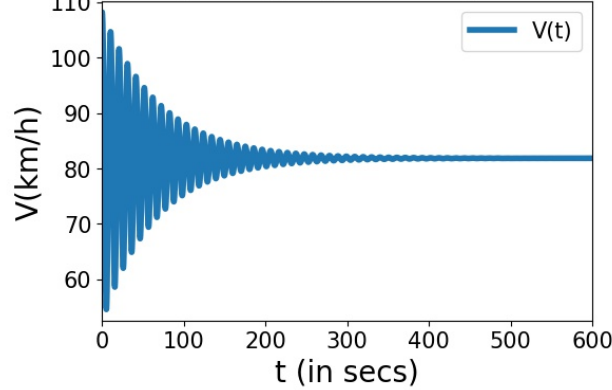


Figure 2.3: **Velocity vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$

Apparently, the numerical solutions obtained from Eulers method demonstrates the characteristic of the solutions. Both V and θ have damping and oscillating natures and the values eventually approach equilibrium as steady state solutions. The analytical procedure to be included in the succeeding section will show how the equilibrium values are influenced by terms in the governing equations, as well as how to determine the frequency and period of oscillation of the solutions.

$$dV/dt = (-C_d V^2 - mg \sin(\theta))/m, \quad d\theta/dt = (C_L V^2 - mg \cos(\theta))/(mV)$$

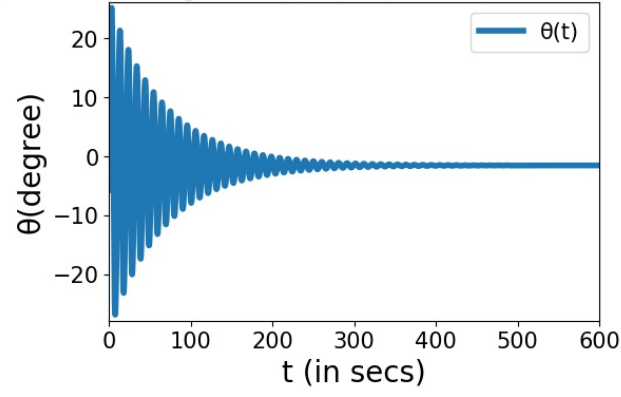


Figure 2.4: **Angle vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48 \cdot 5$, $C_D = 0.04 \cdot 5$

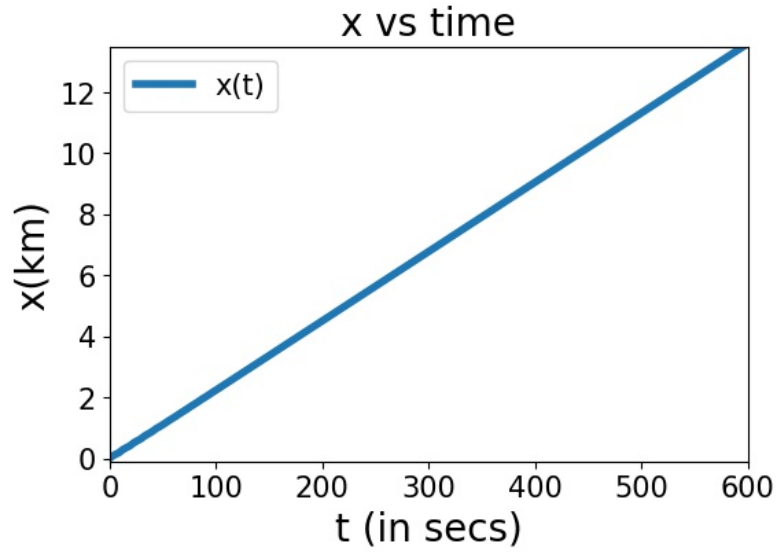


Figure 2.5: **Horizontal distance vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48 \cdot 5$, $C_D = 0.04 \cdot 5$

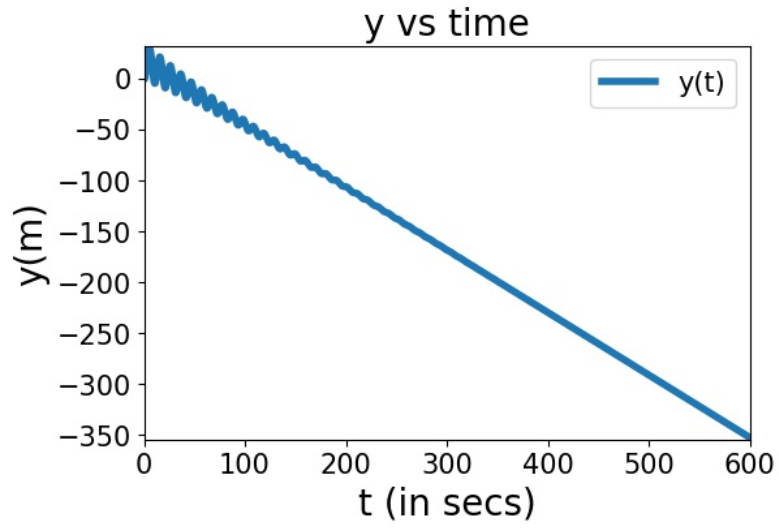


Figure 2.6: **Vertical distance vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48 \cdot 5$, $C_D = 0.04 \cdot 5$

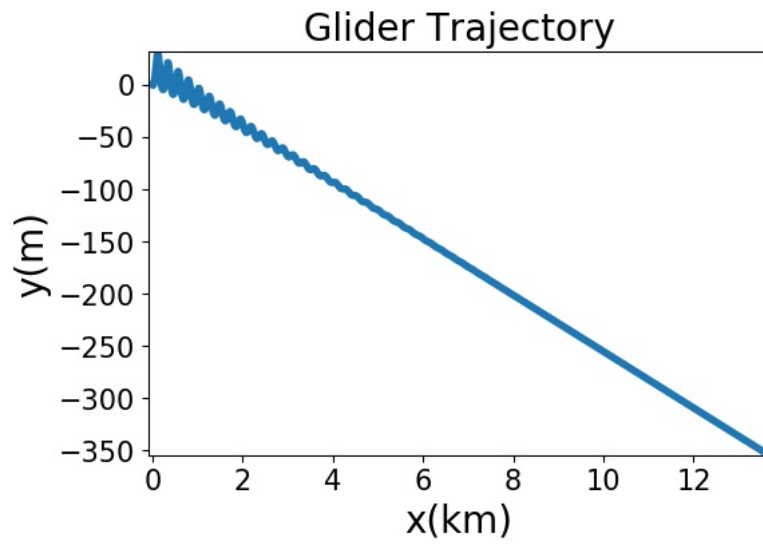


Figure 2.7: **Trajectory of glider's flight.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48 \cdot 5$, $C_D = 0.04 \cdot 5$

2.3 Phase plane and evaluation of mathematical model's result

The advantage of Eulers method allows to quickly obtain solutions of the system of differential equations of a gliders mathematical model. However, precise determination of values can only be evaluated by means of analytical method. Using the linearization technique, phase plane of equilibrium solutions and their characteristics were determined.

$$V_{eq} = \sqrt{\frac{mg \cos(\theta)}{C_L}} \quad (2.5)$$

$$\theta_{eq} = \arctan\left(\frac{C_D}{C_L}\right) \quad (2.6)$$

θ_{eq} depends on the ratio of C_D and C_L and it should be the same with optimal ratio $\frac{L}{D} = \frac{1}{37}$ of the chosen glider provided above. Whereas, V_{eq} depends on the values of θ and C_L given that the mass of the glider is provided.

$$J = \begin{bmatrix} \frac{-2C_D V_{eq}}{m} & -g \cos(\theta_{eq}) \\ \frac{C_L}{m} + \frac{g \cos(\theta_{eq})}{V_{eq}^2} & \frac{g}{V_{eq}} \sin(\theta_{eq}) \end{bmatrix} \quad (2.7)$$

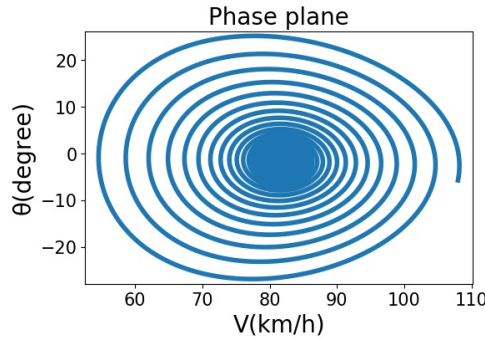


Figure 2.8: **Phase plane of equilibrium point.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$

Above is the Jacobian matrix for the system of differential equations 2.1 and 2.2. Based on the graphs obtained using Eulers method in Python program, it was expected the eigenvalues of the Jacobian Matrix to be complex numbers with negative real parts. Consequently, the oscillation frequency and period

could easily be determined. Figure 2.8 confirms the complex nature of the Jacobian matrix's eigenvalues.

2.4 Conclusion

Being able to quickly assess and interpret the basic system of governing equations of the gliders mathematical model, it was concluded that additional refinement should be implemented to give a more realistic replication of actual gliders flight. Consequently, suggestions were made to promote improvement of the mathematical modelling of the glider.

Chapter 3

Inclusion of dependency of lift and drag coefficients on angle of attack

The coefficients of lift and drag in aerodynamic term are not constants but depend on the angle of attack of the glider. In this chapter, the varying nature of these two terms shall be explored and embedded in the equations of motion of the gliders mathematical model.

3.1 Research on functions of lift and drag

The dependency of lift and drag on angle of attack can be expressed by means of graphs. NACA file of airfoil is used and can be determined through website: <http://airfoiltools.com/search/index>

From the specifications of the chosen glider Cirrus 75, the airfoil type was provided and its NACA file can be found. To simply put, graphs of C_l and C_d against angle Theta are provided once the airfoil type is determined, as shown below:

The information above was obtained with the assumption of Reynolds number $Re = 1.10^6$ (general flow case).

Using the trend feature within Excel file, approximated functions are evaluated to fit the data provided from the NACA file of the chosen airfoil. Subsequently, the approximate functions are substituted into the differential equations 2.1 and 2.2. It is important that the information of maximum and minimum values of C_L and C_D should be included within the evaluated functions.

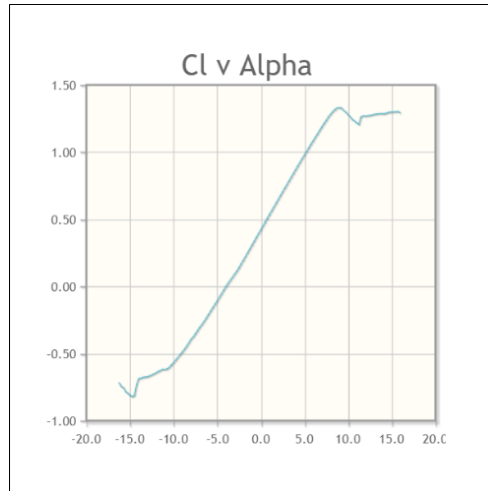


Figure 3.1: **Coefficient of lift vs angle.** (FX 66-17AII-182 AIRFOIL (fx6617a2-il), n.d.)

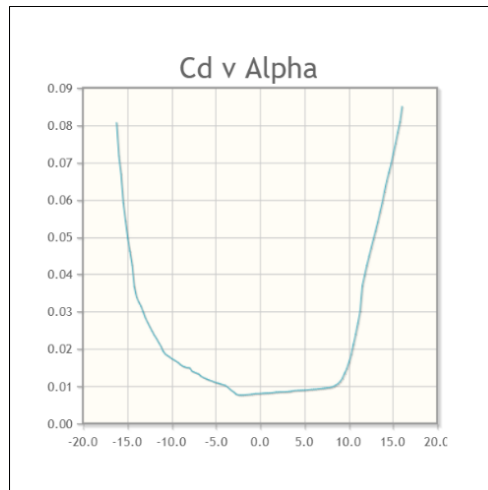


Figure 3.2: **Coefficient of drag vs angle.** FX 66-17AII-182 AIRFOIL (fx6617a2-il), n.d.)

$$C_D(\theta) = 12.086\theta^4 + 0.8489\theta^3 - 0.036\theta^2 - 0.0141\theta + 0.0086 \quad (3.1)$$

$$C_L(\theta) = -38.023\theta^3 - 2.6844\theta^2 + 6.4598\theta + 0.4477 \quad (3.2)$$

Information regarding the values of C_L and C_D against θ can be found in Appendix D.

3.2 Numerical simulation of glider's model

$$dV/dt = (-C_d(\theta)V^2 - mg\sin(\theta))/m, \quad d\theta/dt = (C_l(\theta)V^2 - mg\cos(\theta))/(mV)$$

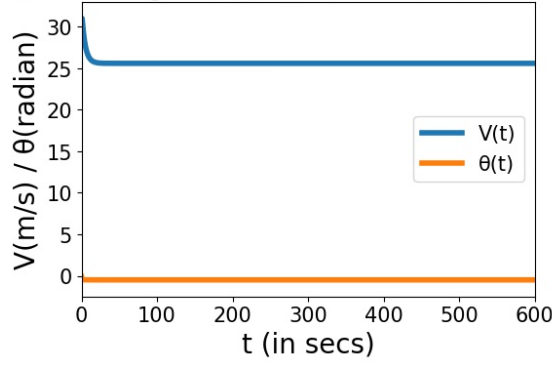


Figure 3.3: **Variations in velocity and angle with time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L(\theta)*5$, $C_D(\theta)*5$

$$dV/dt = (-C_d(\theta)V^2 - mg\sin(\theta))/m, \quad d\theta/dt = (C_l(\theta)V^2 - mg\cos(\theta))/(mV)$$

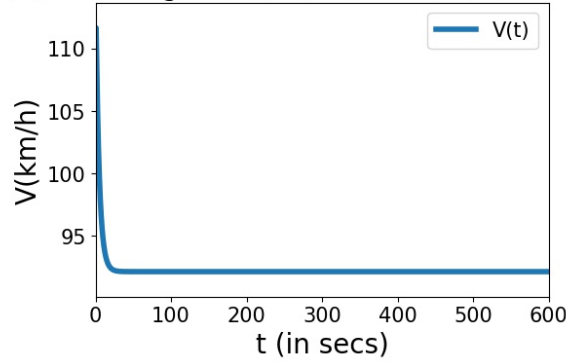


Figure 3.4: **Velocity vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L(\theta)*5$, $C_D(\theta)*5$

Through brief observation, it is to be seen that the solutions do reach a more realistic visualization of the gliders flight trajectory, without oscillation. Both V and θ approach equilibrium values after a long steady flight.

$$dV/dt = (-C_d(\theta)V^2 - mg\sin(\theta))/m, d\theta/dt = (C_l(\theta)V^2 - mg\cos(\theta))/(mV)$$

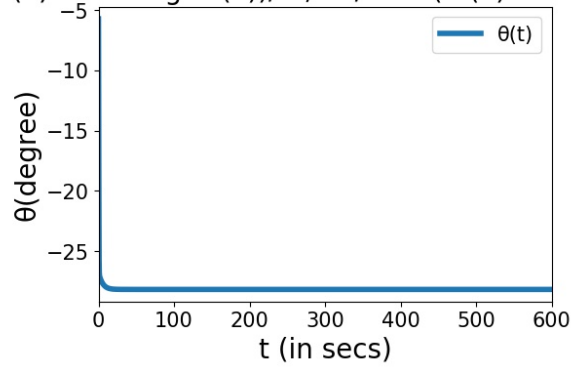


Figure 3.5: **Angle vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L(\theta)*5$, $C_D(\theta)*5$

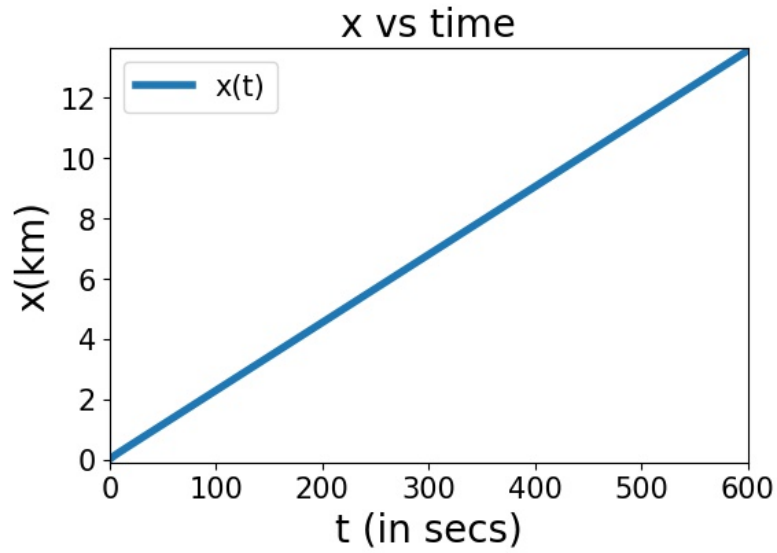


Figure 3.6: **Horizontal distance vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L(\theta)*5$, $C_D(\theta)*5$

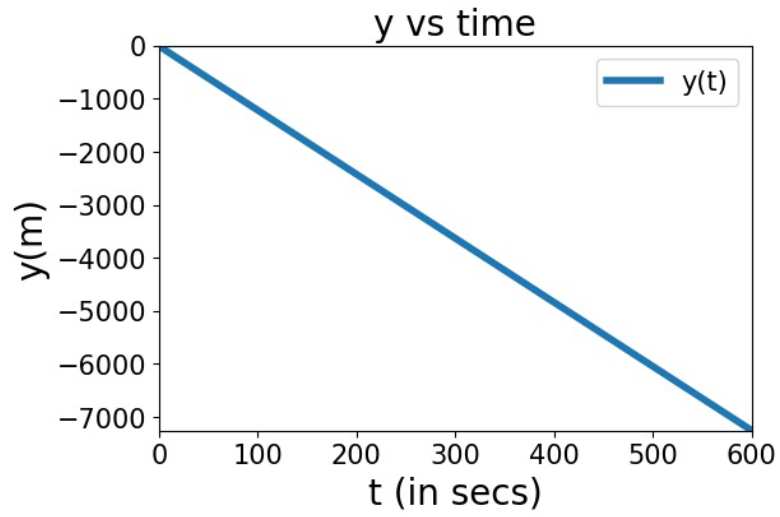


Figure 3.7: **Vertical distance vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$,
 $C_L(\theta)*5$, $C_D(\theta)*5$

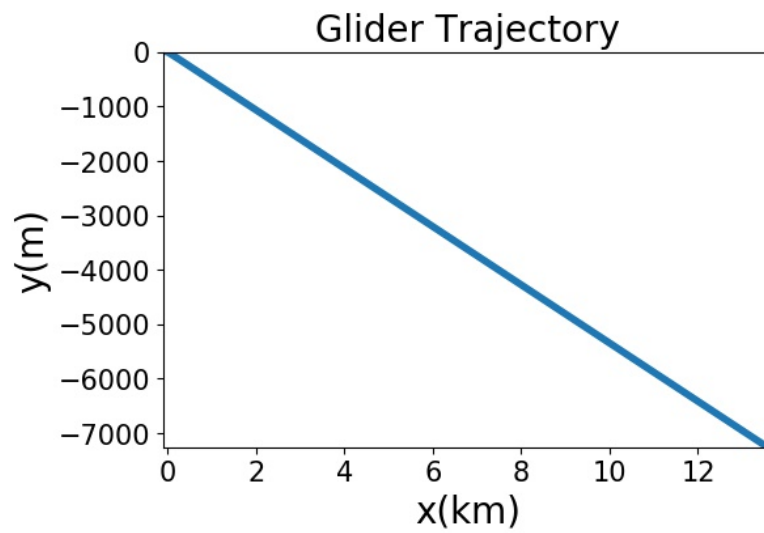


Figure 3.8: **Trajectory of glider's flight.** $V(0) = 30$ m/s, $\theta(0) = -0.1$,
 $C_L(\theta)x5$, $C_D(\theta)*5$

3.3 Interpretation of solution of mathematical model

Investigate separately the solutions of V and θ , the glider maintains a constant velocity in steady state at roughly 92.15 km/h, which is acceptable. Meanwhile, the gliders orientation reaches a constant value of approximately -28.18 degrees. This value, despite being reasonable that the glider should point downwards during its flight, the value is significantly larger compared to the chosen gliders optimal ratio $\frac{L}{D} = \frac{1}{37}$.

3.4 Conclusion

Despite being able to replicate the varying nature of C_D and C_L depending on angle of attack, this process is complicated and it remains to be difficult due to the fact of polynomial approximated equations. Since the highest power values is 4, linearization method cannot be employed to calculate the solution analytically, and thus it is impossible to evaluate the relationship of the solutions with the different expressions of drag and lift coefficients.

Chapter 4

Glider's behaviour during wind

The preceding chapters covers the basic model of the glider covering the first modeling cycle defining the drag and the lift coefficients of the glider, along with the further research on the two coefficients. The adopted values of the drag and lift coefficients as discussed early are 0.04 and 1.48 respectively. Moreover, the preceding chapters show the flight is oscillating in nature. This chapter will cover the effect of the wind on the gliders flight and how its oscillation is influenced by wind.

In this chapter, the possible effects of the wind on the glider are discussed, followed by the modification of the differential equation of the velocity due to the wind. After that, Eulers method will be applied for the gliders system of governing equations to study the behavior of the glider during wind. Finally, the model will be validated with the analytical solution for the differential equation of velocity and angle.

It is noted that due to the time constraint during the project study, only the effect of the wind (limited/permissible wind speed) against the glider is discussed in this chapter.

4.1 Effect of wind during flight

Throughout the duration of a gliders flight, the glider is majorly influenced by the factor air/wind. On the one hand, wind blowing in the same direction of the flight will accelerate the velocity. However, excess wind speed may lead to higher speed of the flight beyond its comfort, which must be avoided for safe flight. On the other hand, wind blowing against the flight will retard the

gliders motion. Again, excess wind speed in this case may change the direction of flight which is generally not desirable.

4.2 Mathematical model considering wind effect

As discussed early, the wind affects the speed of the flight. Therefore, modification is made only to equation 2.1, assuming the wind speed remains constant with time and elevation. In reality, this is rarely the case; however, for certain range of the elevation it can be assumed to be constant. The modified equation is presented in equation 4.1.

$$m \frac{dV}{dt} = -C_D V^2 - mg \sin(\theta) + ma \quad (4.1)$$

Where, a = constant wind speed acting on the flight in m/s and positive value means the flow (wind) in the direction of the flight.

4.3 Simulation of glider system

The Glider system has been analysed with $V(0) = 30$ m/s, $\theta(0) = -0.1$ radians, drag coefficient 0.04, lift coefficient 1.48 and wind speed of 1.4 m/s against the flight. The parameters are same as selected or as concluded in previous chapters except for the wind speed. The wind speed 1.4 m/s is chosen during the study to account the typical speed of 5 km/h of wind at Nepal at this period (June/July). The equations used to simulate the Glider systems are equation 4.1, 2.2, 2.3 and 2.4. Eulers method is used to simulate the systems which is the method early used during this study.

The result of simulation is presented from Figure 4.1 to Figure 4.3.

Figure 4.1 shows that the variation in velocity of flight with time has the nature of bounded and damping oscillation. The model oscillates at around 22.57 m/s. In contrast to basic model of glider (discussed in early chapter), the damping of oscillation is quick which is due to the wind.

Figure 4.2 shows that the variation in angle of flight with time has the nature of bounded and damping oscillation similar to the graph of velocity. The model oscillates at around -0.17017 radians. In contrast to basic model of glider, the damping of oscillation is quick which is due to the wind.

Figure 4.3 shows that there is negligible oscillation in vertical position compare to early study at start of flight. The distance traveled by the flight in horizontal

$$dV/dt = (-C_d V^2 - mg \sin(\theta) + ma)/m, \quad d\theta/dt = (C_l V^2 - mg \cos(\theta))/(mV)$$

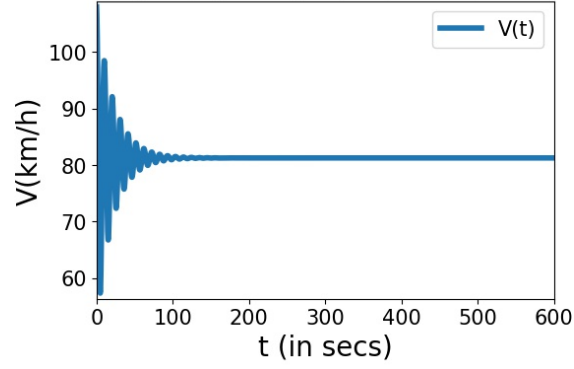


Figure 4.1: **Velocity vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$ and $a = -1.4$ m/s

$$dV/dt = (-C_d V^2 - mg \sin(\theta) + ma)/m, \quad d\theta/dt = (C_l V^2 - mg \cos(\theta))/(mV)$$

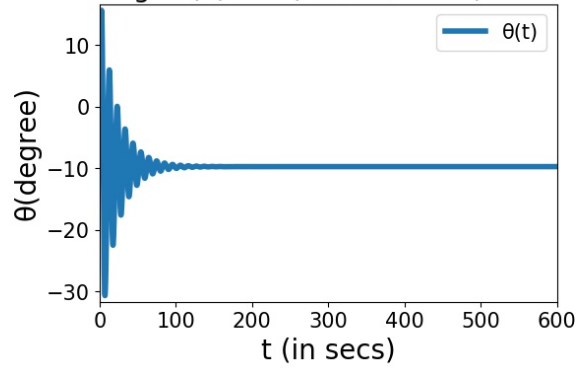


Figure 4.2: **Angle vs time.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$ and $a = -1.4$ m/s

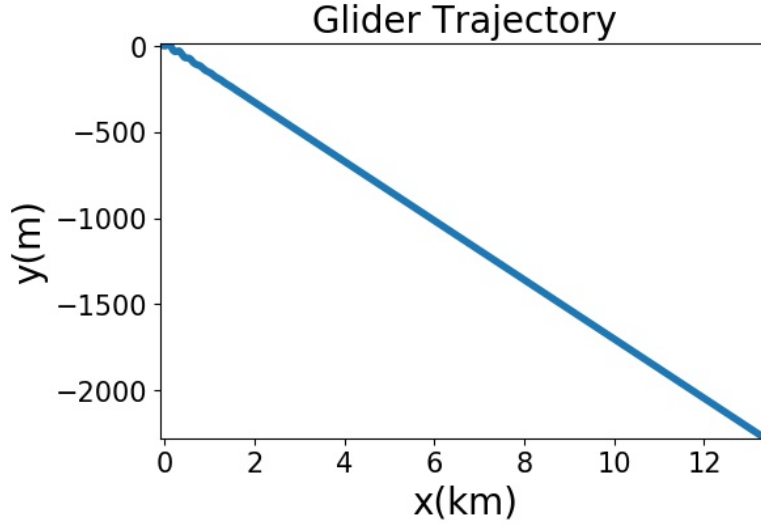


Figure 4.3: **Trajectory of glider's flight.** $V(0) = 30$ m/s, $\theta(0) = -0.1$, $C_L = 1.48*5$, $C_D = 0.04*5$ and $a = -1.4$ m/s

direction is about 13,300 m and drop in height of about 2300 m in 600 s.

4.4 Validation of mathematical model

For the validation of the model, solution of differential equations was obtained analytically. The analytical solution for the modified model is computed from equation 2.3 for the velocity at the equilibrium point and from equation 6.1 for angle of attack at the equilibrium point. The solution is presented symbolically as follow:

$$\begin{cases} V_{eq} = \sqrt{\frac{mg \cos(\theta)}{C_L}} \\ \theta_{eq} = \arctan\left(\frac{-C_D V^2 + ma}{C_L V^2}\right) \end{cases} \quad (4.2)$$

The above equations are solved using iteration process in Excel and for the given condition, i.e. the parameters used in simulation, velocity and angle at equilibrium point are computed to be 22.5731 m/s and -0.170167 radians respectively. These values are the same as the numerically computed values from the simulation. Hence, the modified model is valid.

4.5 Conclusion

The basic mathematical model validated during early study is modified to account the effect of the wind. The early concluded value of drag and lift coefficient is used in modified model. Only one differential equation is modified considering the effect of the wind on the velocity of flight only. The modified differential equation along with other equations defined preceedingly are solved using Eulers method in Python. The obtained numerical solution is validated against analytical solution of the differential equation evaluated in Excel. The result shows that the trajectory is more plausible than the basic model as there is small number of oscillation with low amplitude compare to the basic model.

Chapter 5

Conclusions

The two extension models provided in previous chapters are examples of refining the basic system of governing differential equations of the glider's mathematical model. Based on the observations of previous work, suggestions were made to attempt inclusion of the dependency of C_D and C_L on angle of attack and the effect of wind.

Given the specifications of the chosen glider, the type of airfoil was used to obtain NACA file which contains the values of C_D and C_L for each value of θ . Despite improving the mathematical model, the solutions obtain were only practice for value of the glider's velocity and not of the glider's angle of orientation. Consequently, it remains to be a complicated process to include the varying nature of the coefficients of lift and drag in the system of differential equations of the glider's model.

On the other hand, the effect of wind was also investigate during this work. Though it was not an extensive model, the fundamental idea of the influence of wind to the glider's flight was sufficiently illustrate in terms of an additional variable within the differential equation $\frac{dV}{dt}$. Solutions were calculated numerically using Euler's method and was verified by analytical method to ensure the validity of solutions.

In general, it is apparent that eventhough the refinement steps illustrated above were able to replicate better the reality, the process involved is still much complicated depending on what criteria the authors concentrated on. By taking into account the dependency of coefficients of lift and drag on the angle of attack, the simulated result showed a more steady and stable flight without oscillating behaviour. On the contrary, the inclusion of wind effect, though did not reduce much of the oscillating characteristic of the preliminary mathematical modelling cycle, was still able to show the damping effect to the original solutions as expected. Whether one idea shall be more feasible

comparing to the other, it is up to the author to decide which case is best suited for a particular investigation of the glider's trajectory.

Bibliography

[1] *Cirrus 75 Specifications*. Retrieved from Standard Cirrus:

[2] *Glider model*. Retrieved from DelftX: MathMod1x Mathematical Modelling Basics:

<https://courses.edx.org/courses/course-v1:DelftX+MathMod1x+2T2017/course/>

[3] *FX 66-17AII-182 AIRFOIL graphs*. Retrieved from Airfoil Tools:

<http://airfoiltools.com/polar/details?polar=xf-fx6617a2-il-1000000>

Appendix A

Preliminary simulation using Euler's method - Python programming code

```
# Program      : Euler's method for a system
# Author       : MOOC team Mathematical Modelling Basics
# Created      : May, 2017
# Edited       : Anil and Vu Le

import numpy as np
import matplotlib.pyplot as plt

print("Solution for  $dV/dt = (-C_d V^2 - mg \sin(\alpha))/m$ ,  $d/dt = (C_l V^2 - mg \cos(\alpha))/(mV)$ ")

# Constant
m = 390                                # mass of glider in kg
g = 9.81                              # Acceleration due to gravity in  $m/s^2$ 
Cd = 0.04*5                            # Drag Coefficient; the highest value of that r
Cl = 1.48*5                            # Lift Coefficient; the highest value of the ran

#time steps
t_init = 0                            # initial time
t_end = 600                           # stopping time
t_step = 64                           #reciprocal of time step in second for first time

#Initialization for 1/t_step second
Dt = 1/t_step                          # timestep Delta t
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps
```

```

#Boundary condition
V_init = 30 # initial velocity of V
_init = -0.1 # initial angle of
x_init = 0 # initial angle of x
y_init = 0 # initial angle of y

#Define array
X = np.zeros(4) # create space for current X=[V,]^T
dXd_t = np.zeros(4) # create space for current derivative
t_arr = np.zeros(n_steps + 1) # create a storage array for t
X_arr = np.zeros((4,n_steps+1)) # create a storage array for X=[V,]^T
t_arr[0] = t_init # add the initial t to the storage array
X_arr[0,0] = V_init # add the initial V to the storage array
X_arr[1,0] = _init # add the initial to the storage array
X_arr[2,0] = x_init # add the initial x to the storage array
X_arr[3,0] = y_init # add the initial y to the storage array

# Euler's method
for i in range (1, n_steps + 1):
    t = t_arr[i-1] # load the time
    V = X_arr[0,i-1] # load the value of V
    = X_arr[1,i-1] # load the value of
    x = X_arr[2,i-1] # load the value of x
    y = X_arr[3,i-1] # load the value of y
    X[0] = V # fill current state vector X=[V,]^T
    X[1] =
    X[2] = x
    X[3] = y
    dVdt = (-Cd*V**2 - m*g*np.sin())/m # calculate the derivative dV/dt
    ddt = (Cl*V**2 - m*g*np.cos())/(m*V) # calculate the derivative d/dt
    dxdt = V*np.cos() # calculate the derivative dx/dt
    dydt = V*np.sin() # calculate the derivative dy/dt
    dXd_t[0] = dVdt # fill derivative vector dX/dt
    dXd_t[1] = ddt
    dXd_t[2] = dxdt
    dXd_t[3] = dydt
    Xnew = X + Dt*dXd_t # calculate X on next time step
    X_arr[:,i] = Xnew # store Xnew
    t_arr[i] = t + Dt # store new t-value
    #print(t_arr[i],X_arr[:,i]) # store new t-value

# Plot the results
fig = plt.figure()
plt.plot(t_arr, X_arr[0,:], linewidth = 4, label="V(t)") # plot V vs. time
plt.plot(t_arr, X_arr[1,:], linewidth = 4, label="(t)") # plot vs. time

```

```

plt.title('dV/dt = (-CdV^2 mgsin())/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize = 20)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(m/s) / (radian)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])-2, max(X_arr[0,:])+2]) # set the r

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Task_3.5_Glider Speed and Angle.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Velocity
fig1 = plt.figure()
plt.plot(t_arr, X_arr[0,:]*3.6, linewidth = 4, label="V(t)") # plot V vs. time

plt.title('dV/dt = (-CdV^2 mgsin())/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize = 20)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(km/h)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[0,:])*3.6-2, max(X_arr[0,:])*3.6+2]) # s

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[0,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig1.savefig('Task_3.5_Glider Speed.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Angle
fig2 = plt.figure()
plt.plot(t_arr, X_arr[1,:]*180/np.pi, linewidth = 4, label="(t)") # plot vs. time

plt.title('dV/dt = (-CdV^2 mgsin())/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize = 20)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('(degree)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize

```

```

plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])*180/np.pi-1, max(X_arr[1,:])*180/np.pi+1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[1,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig2.savefig('Task_3.5_Glider Angle.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the x vs time
fig3 = plt.figure()
plt.plot(t_arr, X_arr[2,:]/1000, linewidth = 4, label="x(t)") # plot x vs. time

plt.title('x vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('x(km)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000-0.1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[2,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig3.savefig('Task_3.5_Glider x vs t.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the y vs time
fig4 = plt.figure()
plt.plot(t_arr, X_arr[3:], linewidth = 4, label="y(t)") # plot y vs. time

plt.title('y vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[3,:])-2, max(X_arr[3,:])+2]) # set the r

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

```



```

print (t_arr[t_end*t_step],X_arr[3,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig4.savefig('Task_3.5_Glider y vs t.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Trejectory
fig5 = plt.figure()
plt.plot(X_arr[2,:]/1000, X_arr[3,:], linewidth = 4) # plot y vs. x

plt.title('Glider Trajectory', fontsize = 20) # set title
plt.xlabel('x(km)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000+0.1, min(X_arr[3,:])-2, max

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step], X_arr[0,t_end*t_step],X_arr[1,t_end*t_step],X_arr[2,t_end

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig5.savefig('Task_3.5_Trejectory.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the phase plane
fig5 = plt.figure()
plt.plot(X_arr[0,:]*3.6, X_arr[1,:]*180/np.pi, linewidth = 4) # plot y vs. x

plt.title('Phase plane', fontsize = 20) # set title
plt.xlabel('V(km/h)', fontsize = 20) # name of horizontal axis
plt.ylabel('(degree)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([min(X_arr[0,:])*3.6-2, max(X_arr[0,:])*3.6+2, min(X_arr[1,:])*180/np.pi-1,

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step], X_arr[0,t_end*t_step],X_arr[1,t_end*t_step],X_arr[2,t_end

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig5.savefig('Task_3.5_Phase_Plane.jpg', dpi=fig.dpi, bbox_inches = "tight")

```

Appendix B

Dependency of C_D and C_L on angle of attack simulation - Python programming code

```
# Program      : Euler's method for a system
# Author       : MOOC team Mathematical Modelling Basics
# Created      : May, 2017
# Edited       : Anil and Vu Le

import numpy as np
import matplotlib.pyplot as plt

print("Solution for  $dV/dt = (-C_d V^2 \sin \alpha)/m$ ,  $d\alpha/dt = (C_L V^2 \cos \alpha)/(mV)$ ")

# Constant
m = 390                                # mass of glider in kg
g = 9.81                              # Acceleration due to gravity in  $m/s^2$ 
Cd = 0.04*5                            # Drag Coefficient; the highest value of that r
Cl = 1.48*5                            # Lift Coefficient; the highest value of the ran

#time steps
t_init = 0                            # initial time
t_end = 600                           # stopping time
t_step = 64                           #reciprocal of time step in second for first time

#Initialization for 1/t_step second
Dt = 1/t_step                          # timestep Delta t
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps
```

```

#Boundary condition
V_init = 30 # initial velocity of V
_init = -0.1 # initial angle of
x_init = 0 # initial angle of x
y_init = 0 # initial angle of y

#Define array
X = np.zeros(4) # create space for current X=[V,]^T
dXd_t = np.zeros(4) # create space for current derivative
t_arr = np.zeros(n_steps + 1) # create a storage array for t
X_arr = np.zeros((4,n_steps+1)) # create a storage array for X=[V,]^T
t_arr[0] = t_init # add the initial t to the storage array
X_arr[0,0] = V_init # add the initial V to the storage array
X_arr[1,0] = _init # add the initial to the storage array
X_arr[2,0] = x_init # add the initial x to the storage array
X_arr[3,0] = y_init # add the initial y to the storage array

# Euler's method
for i in range (1, n_steps + 1):
    t = t_arr[i-1] # load the time
    V = X_arr[0,i-1] # load the value of V
    = X_arr[1,i-1] # load the value of
    x = X_arr[2,i-1] # load the value of x
    y = X_arr[3,i-1] # load the value of y
    X[0] = V # fill current state vector X=[V,]^T
    X[1] =
    X[2] = x
    X[3] = y
    dVdt = (-4.5*(12.086***4+0.8489***3-0.036***2-0.0141**+0.0086)*V**2 - m*g*np.sin())/m
    ddt = (4.5*(-38.023***3-2.6844***2+6.4598**+0.4477)*V**2 - m*g*np.cos())/(m*V)
    dxdt = V*np.cos() # calculate the derivative dx/dt
    dydt = V*np.sin() # calculate the derivative dx/dt
    dXd_t[0] = dVdt # fill derivative vector dX/dt
    dXd_t[1] = ddt
    dXd_t[2] = dxdt
    dXd_t[3] = dydt
    Xnew = X + Dt*dXd_t # calculate X on next time step
    X_arr[:,i] = Xnew # store Xnew
    t_arr[i] = t + Dt # store new t-value
    #print(t_arr[i],X_arr[:,i]) # store new t-value

# Plot the results
fig = plt.figure()
plt.plot(t_arr, X_arr[0,:], linewidth = 4, label="V(t)") # plot V vs. time
plt.plot(t_arr, X_arr[1,:], linewidth = 4, label="(t)") # plot vs. time

```

```

plt.title('dV/dt = (-Cd()V^2 mgsin())/m, d/dt = (Cl()V^2 mgcos())/(mV)', fontsize = 12)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(m/s) / (radian)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])-2, max(X_arr[0,:])+2]) # set the range

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Task_3.5_Glider Speed and Angle_modified.jpg', dpi=fig.dpi, bbox_inches='tight')

# Plot the results Velocity
fig1 = plt.figure()
plt.plot(t_arr, X_arr[0,:]*3.6, linewidth = 4, label="V(t)") # plot V vs. time

plt.title('dV/dt = (-Cd()V^2 mgsin())/m, d/dt = (Cl()V^2 mgcos())/(mV)', fontsize = 12)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(km/h)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[0,:])*3.6-2, max(X_arr[0,:])*3.6+2]) # set the range

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[0,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig1.savefig('Task_3.5_Glider Speed_modified.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Angle
fig2 = plt.figure()
plt.plot(t_arr, X_arr[1,:]*180/np.pi, linewidth = 4, label="(t)") # plot angle vs. time

plt.title('dV/dt = (-Cd()V^2 mgsin())/m, d/dt = (Cl()V^2 mgcos())/(mV)', fontsize = 12)
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('(degree)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize

```

```

plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])*180/np.pi-1, max(X_arr[1,:])*180/np.pi+1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[1,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig2.savefig('Task_3.5_Glider Angle_modified.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the x vs time
fig3 = plt.figure()
plt.plot(t_arr, X_arr[2,:]/1000, linewidth = 4, label="x(t)") # plot x vs. time

plt.title('x vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('x(km)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000+0.1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[2,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig3.savefig('Task_3.5_Glider x vs t_modified.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the y vs time
fig4 = plt.figure()
plt.plot(t_arr, X_arr[3:], linewidth = 4, label="y(t)") # plot y vs. time

plt.title('y vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[3,:])-2, max(X_arr[3,:])+2]) # set the r

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

```

```

print (t_arr[t_end*t_step],X_arr[3,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig4.savefig('Task_3.5_Glider y vs t_modified.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Trejectory
fig5 = plt.figure()
plt.plot(X_arr[2,:]/1000, X_arr[3,:], linewidth = 4) # plot y vs. x

plt.title('Glider Trajectory', fontsize = 20) # set title
plt.xlabel('x(km)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000+0.1, min(X_arr[3,:])-2, max(X_arr[3,:])])

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step], X_arr[0,t_end*t_step]*3.6,X_arr[1,t_end*t_step]*180/np.pi)

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig5.savefig('Task_3.5_Trejectory_modified.jpg', dpi=fig.dpi, bbox_inches = "tight")

```

Appendix C

Effect of wind on the flight of glider - Python programming code

```
# Program      : Euler's method for a system
# Author       : MOOC team Mathematical Modelling Basics
# Created      : May, 2017
# Edited       : Anil and Vu Le

import numpy as np
import matplotlib.pyplot as plt

print("Solution for  $dV/dt = (-C_d V^2 - mg \sin(\alpha))/m$ ,  $d\alpha/dt = (C_l V^2 - mg \cos(\alpha))/(mV)$ ")

# Constant
m = 390                                # mass of glider in kg
g = 9.81                               # Acceleration due to gravity in  $m/s^2$ 
Cd = 0.04*5                            # Drag Coefficient; the highest value of that range
Cl = 1.48*5                             # Lift Coefficient; the highest value of the range
a=-1.4                                 # wind speed in m/s, negative values mean wind from behind

#time steps
t_init = 0                             # initial time
t_end = 600                             # stopping time
t_step = 64                             #reciprocal of time step in second for first time step

#Initialization for 1/t_step second
Dt = 1/t_step                           # timestep Delta t
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps
```

```

#Boundary condition
V_init = 30                                # initial velocity of V
_init = -0.1                               # initial angle of
x_init = 0 # initial angle of x
y_init = 0 # initial angle of y

#Define array
X = np.zeros(4)                            # create space for current X=[V,]^T
dXd_t = np.zeros(4)                       # create space for current derivative
t_arr = np.zeros(n_steps + 1)             # create a storage array for t
X_arr = np.zeros((4,n_steps+1))           # create a storage array for X=[V,]^T
t_arr[0] = t_init                         # add the initial t to the storage array
X_arr[0,0] = V_init                       # add the initial V to the storage array
X_arr[1,0] = _init                        # add the initial  to the storage array
X_arr[2,0] = x_init                       # add the initial x to the storage array
X_arr[3,0] = y_init                       # add the initial y to the storage array

# Euler's method
for i in range (1, n_steps + 1):
    t = t_arr[i-1]                        # load the time
    V = X_arr[0,i-1]                      # load the value of V
    = X_arr[1,i-1]                        # load the value of
    x = X_arr[2,i-1]                      # load the value of x
    y = X_arr[3,i-1]                      # load the value of y
    X[0] = V                             # fill current state vector X=[V,]^T
    X[1] =
    X[2] = x
    X[3] = y
    dVdt = (-Cd*V**2 - m*g*np.sin()+m*a)/m # calculate the derivative dV/dt
    ddt = (Cl*V**2 - m*g*np.cos())/(m*V)    # calculate the derivative d/dt
    dxdt = V*np.cos() # calculate the derivative dx/dt
    dydt = V*np.sin() # calculate the derivative dy/dt
    dXd_t[0] = dVdt                        # fill derivative vector dX/dt
    dXd_t[1] = ddt
    dXd_t[2] = dxdt
    dXd_t[3] = dydt
    Xnew = X + Dt*dXd_t                   # calculate X on next time step
    X_arr[:,i] = Xnew                     # store Xnew
    t_arr[i] = t + Dt                     # store new t-value
    #print(t_arr[i],X_arr[:,i])           # store new t-value

# Plot the results
fig = plt.figure()
plt.plot(t_arr, X_arr[0,:], linewidth = 4, label="V(t)") # plot V vs. time

```



```

plt.plot(t_arr, X_arr[1,:], linewidth = 4, label="(t)") # plot vs. time

plt.title('dV/dt = (-CdV^2 mgsin()+ma)/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize =
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(m/s) / (radian)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])-2, max(X_arr[0,:])+2]) # set the r

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig.savefig('Task_5_Glider Speed and Angle.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Velocity
fig1 = plt.figure()
plt.plot(t_arr, X_arr[0,:]*3.6, linewidth = 4, label="V(t)") # plot V vs. time

plt.title('dV/dt = (-CdV^2 mgsin()+ma)/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize =
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('V(km/h)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([0, t_end, min(X_arr[0,:])*3.6-1, max(X_arr[0,:])*3.6+1]) # s

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[0,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig1.savefig('Task_5_Glider Speed.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Angle
fig2 = plt.figure()
plt.plot(t_arr, X_arr[1,:]*180/np.pi, linewidth = 4, label="(t)") # plot vs. time

plt.title('dV/dt = (-CdV^2 mgsin()+ma)/m, d/dt = (ClV^2 mgcos())/(mV)', fontsize =
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('(degree)', fontsize = 20) # name of vertical axis

```

```

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[1,:])*180/np.pi-1, max(X_arr[1,:])*180/np.pi+1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[1,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig2.savefig('Task_5_Glider Angle.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the x vs time
fig3 = plt.figure()
plt.plot(t_arr, X_arr[2,:]/1000, linewidth = 4, label="x(t)") # plot x vs. time

plt.title('x vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('x(km)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000+0.1])

plt.legend(fontsize=15)           # show the legend
plt.show()                        # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[2,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig3.savefig('Task_5_Glider x vs t.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the y vs time
fig4 = plt.figure()
plt.plot(t_arr, X_arr[3,:], linewidth = 4, label="y(t)") # plot y vs. time

plt.title('y vs time', fontsize = 20) # set title
plt.xlabel('t (in secs)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15)          # adjust the fontsize
plt.yticks(fontsize = 15)          # adjust the fontsize
plt.axis([0, t_end, min(X_arr[3,:])-2, max(X_arr[3,:])+2]) # set the r

plt.legend(fontsize=15)           # show the legend

```

```

plt.show()                                # necessary for some platforms

print (t_arr[t_end*t_step],X_arr[3,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig4.savefig('Task_5_Glider y vs t.jpg', dpi=fig.dpi, bbox_inches = "tight")

# Plot the results Trejectory
fig5 = plt.figure()
plt.plot(X_arr[2,:]/1000, X_arr[3,:], linewidth = 4) # plot y vs. x

plt.title('Glider Trajectory', fontsize = 20) # set title
plt.xlabel('x(km)', fontsize = 20) # name of horizontal axis
plt.ylabel('y(m)', fontsize = 20) # name of vertical axis

plt.xticks(fontsize = 15) # adjust the fontsize
plt.yticks(fontsize = 15) # adjust the fontsize
plt.axis([min(X_arr[2,:])/1000-0.1, max(X_arr[2,:])/1000+0.1, min(X_arr[3,:])-2, max(X_arr[3,:])+2])

plt.legend(fontsize=15) # show the legend
plt.show() # necessary for some platforms

print (t_arr[t_end*t_step], X_arr[0,t_end*t_step],X_arr[1,t_end*t_step],X_arr[2,t_end*t_step])

# save the figure as .jpg (other formats: png, pdf, svg, (ps, eps))
fig5.savefig('Task_5_Trejectory.jpg', dpi=fig.dpi, bbox_inches = "tight")

```

Appendix D

Values of C_D and C_L against
angle of attack of FX
66-17AII-182 airfoil

alpha - degree	alpha - radian	CL	CD	Cl/Cd
-----	-----	-----	-----	-----
-16.25	-0.2836	-0.7174	0.08076	-8.8831
-16	-0.2793	-0.7465	0.07184	-10.3911
-15.75	-0.2749	-0.7539	0.06698	-11.2556
-15.5	-0.2705	-0.7832	0.05924	-13.2208
-15.25	-0.2662	-0.796	0.05444	-14.6216
-15	-0.2618	-0.8116	0.04976	-16.3103
-14.75	-0.2574	-0.8204	0.04623	-17.7461
-14.5	-0.2531	-0.8124	0.04256	-19.0883
-14.25	-0.2487	-0.7303	0.03668	-19.9100
-14	-0.2443	-0.6861	0.03392	-20.2270
-13.75	-0.2400	-0.6838	0.03258	-20.9883
-13.5	-0.2356	-0.6771	0.03147	-21.5157
-13.25	-0.2313	-0.6742	0.02995	-22.5109
-13	-0.2269	-0.6731	0.0283	-23.7845
-12.75	-0.2225	-0.6656	0.02714	-24.5247
-12.5	-0.2182	-0.6587	0.02593	-25.4030
-12.25	-0.2138	-0.6506	0.02484	-26.1916
-12	-0.2094	-0.6431	0.02369	-27.1465
-11.75	-0.2051	-0.6325	0.02281	-27.7291
-11.5	-0.2007	-0.6251	0.02172	-28.7799
-11.25	-0.1963	-0.6175	0.02075	-29.7590
-11	-0.1920	-0.6193	0.01939	-31.9391
-10.75	-0.1876	-0.6143	0.01865	-32.9383
-10.5	-0.1833	-0.6036	0.01821	-33.1466
-10.25	-0.1789	-0.5851	0.01784	-32.7971
-10	-0.1745	-0.5675	0.01739	-32.6337
-9.75	-0.1702	-0.5476	0.01707	-32.0797
-9.5	-0.1658	-0.5283	0.01668	-31.6727
-9.25	-0.1614	-0.5073	0.01629	-31.1418
-9	-0.1571	-0.4889	0.01572	-31.1005
-8.75	-0.1527	-0.4674	0.01537	-30.4099
-8.5	-0.1484	-0.444	0.01515	-29.3069
-8.25	-0.1440	-0.4196	0.015	-27.9733
-8	-0.1396	-0.3925	0.01496	-26.2366
-7.75	-0.1353	-0.3765	0.01407	-26.7591
-7.5	-0.1309	-0.3523	0.01381	-25.5105
-7.25	-0.1265	-0.3282	0.01354	-24.2393
-7	-0.1222	-0.3033	0.01335	-22.7191
-6.75	-0.1178	-0.2833	0.01276	-22.2022
-6.5	-0.1134	-0.2593	0.0124	-20.9113
-6.25	-0.1091	-0.2343	0.01212	-19.3317
-6	-0.1047	-0.209	0.01186	-17.6223
-5.75	-0.1004	-0.1833	0.01164	-15.7474
-5.5	-0.0960	-0.1573	0.01146	-13.7260
-5.25	-0.0916	-0.1328	0.01116	-11.8996
-5	-0.0873	-0.1073	0.01098	-9.7723
-4.75	-0.0829	-0.0809	0.0108	-7.4907
-4.5	-0.0785	-0.0541	0.01063	-5.0894
-4.25	-0.0742	-0.0274	0.01045	-2.6220

-4	-0.0698	-0.0009	0.01027	-0.0876
-3.75	-0.0654	0.0231	0.00984	2.3476
-3.5	-0.0611	0.0461	0.00935	4.9305
-3.25	-0.0567	0.0685	0.00883	7.7576
-3	-0.0524	0.0922	0.00846	10.8983
-2.75	-0.0480	0.1139	0.00789	14.4360
-2.5	-0.0436	0.14	0.00769	18.2055
-2.25	-0.0393	0.1677	0.00766	21.8930
-2	-0.0349	0.1958	0.00766	25.5614
-1.75	-0.0305	0.2242	0.00771	29.0791
-1.5	-0.0262	0.2526	0.00775	32.5935
-1.25	-0.0218	0.281	0.00781	35.9795
-1	-0.0175	0.3091	0.00787	39.2757
-0.75	-0.0131	0.3374	0.00798	42.2807
-0.5	-0.0087	0.3659	0.00804	45.5100
-0.25	-0.0044	0.3942	0.00803	49.0909
0	0.0000	0.4225	0.00808	52.2896
0.25	0.0044	0.451	0.00812	55.5419
0.5	0.0087	0.4795	0.00817	58.6903
0.75	0.0131	0.5077	0.00818	62.0660
1	0.0175	0.5358	0.0082	65.3415
1.25	0.0218	0.5638	0.00825	68.3394
1.5	0.0262	0.5918	0.00834	70.9592
1.75	0.0305	0.62	0.00843	73.5469
2	0.0349	0.6481	0.00845	76.6982
2.25	0.0393	0.6762	0.00849	79.6466
2.5	0.0436	0.7041	0.00849	82.9329
2.75	0.0480	0.7319	0.00851	86.0047
3	0.0524	0.7596	0.00854	88.9461
3.25	0.0567	0.7873	0.0086	91.5465
3.5	0.0611	0.8149	0.00868	93.8825
3.75	0.0654	0.8426	0.00882	95.5329
4	0.0698	0.8703	0.00887	98.1172
4.25	0.0742	0.8978	0.00891	100.7632
4.5	0.0785	0.9251	0.00892	103.7108
4.75	0.0829	0.9522	0.00895	106.3911
5	0.0873	0.9788	0.00896	109.2411
5.25	0.0916	1.0048	0.00903	111.2735
5.5	0.0960	1.0315	0.0091	113.3516
5.75	0.1004	1.0584	0.00914	115.7987
6	0.1047	1.085	0.00919	118.0631
6.25	0.1091	1.1107	0.0092	120.7283
6.5	0.1134	1.1357	0.0093	122.1183
6.75	0.1178	1.1616	0.00936	124.1026
7	0.1222	1.1877	0.00941	126.2168
7.25	0.1265	1.2129	0.00948	127.9430
7.5	0.1309	1.2359	0.00957	129.1432
7.75	0.1353	1.2607	0.00961	131.1863
8	0.1396	1.2826	0.00973	131.8191
8.25	0.1440	1.3014	0.00996	130.6627
8.5	0.1484	1.3164	0.01032	127.5581

8.75	0.1527	1.3296	0.01066	124.7280
9	0.1571	1.3317	0.01127	118.1633
9.25	0.1614	1.3278	0.01216	109.1941
9.5	0.1658	1.3104	0.0135	97.0667
9.75	0.1702	1.2985	0.01477	87.9147
10	0.1745	1.2803	0.01652	77.5000
10.25	0.1789	1.2633	0.0186	67.9194
10.5	0.1833	1.2434	0.02136	58.2116
10.75	0.1876	1.2313	0.02402	51.2614
11	0.1920	1.2193	0.02691	45.3103
11.25	0.1963	1.2048	0.03024	39.8413
11.5	0.2007	1.2628	0.03701	34.1205
11.75	0.2051	1.2703	0.03971	31.9894
12	0.2094	1.2687	0.04236	29.9504
12.25	0.2138	1.2703	0.04478	28.3676
12.5	0.2182	1.2733	0.04714	27.0110
12.75	0.2225	1.2763	0.0495	25.7838
13	0.2269	1.2807	0.05175	24.7478
13.25	0.2313	1.2835	0.05419	23.6852
13.5	0.2356	1.2858	0.0567	22.6772
13.75	0.2400	1.2876	0.0593	21.7133
14	0.2443	1.2864	0.06223	20.6717
14.25	0.2487	1.2866	0.06508	19.7695
14.5	0.2531	1.2925	0.06731	19.2022
14.75	0.2574	1.2967	0.06971	18.6013
15	0.2618	1.299	0.07238	17.9469
15.25	0.2662	1.2998	0.07523	17.2777
15.5	0.2705	1.3009	0.07811	16.6547
15.75	0.2749	1.3022	0.08094	16.0885
16	0.2793	1.2928	0.08509	15.1933